

University of Pittsburgh

Introduction to Operating Systems
CS 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Quiz 2: due on 3/25
  - Homework 9: due on 3/28
  - Lab 3: due on 4/1
  - Project 3: due on 4/11

Previous lecture …

- Page replacement algorithms

Algorithm	**Comment**
OPT (Optimal)	Not implementable, but useful as a benchmark
NRU (Not Recently Used)	Crude
FIFO (First-In, First Out)	Might throw out useful pages
Second chance	Big improvement over FIFO
Clock	Better implementation of second chance
LRU (Least Recently Used)	Excellent, but hard to implement exactly
NFU (Not Frequently Used)	Poor approximation to LRU
Aging	Good approximation to LRU, efficient to implement
Working Set	Somewhat expensive to implement
WSClock	Implementable version of Working Set

Muddiest Points (Memory)

- Difference between NRU and NFU
- For NRU, what is the way in which it separates the pages into classes in order to pick a victim from a group of the lowest non-empty class
- On the clock for the clock algorithm, what is t and why does it change to 32 on the referenced pages?
- What are we adding to the shift register in the working set replacement algorithm?
- Why is the tracking of a process' working set necessary?
- Which page replacement algorithm is the best (generally)?
- The question on the homework that has the numerical entry answer. I'm not quite sure how to adapt what we discussed in class to that question.
- How the eviction happens in the pages

Muddiest Points (Memory)

- How do we determine tau again?
- On the last slide about working set page, does the page table get updated every clock tick? Or do we only care about evey clock interrupt here? I am a little confused about how is the clock interrupt concept applied to the working set page replacement algorithm.
- What is k in the working set graph?
- in second chance and clock replacement, why do referenced pages have their ref bit set back to 0?
- Do most algorithms need to iterate through all pages (O(N) runtime)?
- I am confused on the details of thrashing
- LRU page replacement

Muddiest Points (Misc.)

- i had a bit of a hard time keeping up with pretty much everything but i think that's just a side effect of coming back from break
- what is it that makes the TLB access fast while being slower for memory PTE access
- when is the midterm grades posted also will there be makeup to get up to 30% back?
- When are TLB entries replaced
- Is "Frame" the same thing as a physical page?
- How is the page table indexed?
- redo a walk through the address translation process with the CPU, MMU, & TLB caching

Problem of the Day

- How to simulate page replacement algorithms
  - FIFO/Clock
  - LRU, OPT

How is modeling done?

- Generate a list of references
  - Artificial (made up)
  - Trace a real workload (set of processes)
- Use an array (or other structure) to track the pages in physical memory at any given time
  - May keep other information per page to help simulate the algorithm (modification time, time when paged in, etc.)
- Run through references, applying the replacement algorithm
- Example: FIFO replacement on reference string 0 1 2 3 0 1 4 0 1 2 3 4
  - Page replacements highlighted in yellow

Page referenced	0	1	2	3	0	1	4	0	1	2	3	4
Youngest page	0	1	2	3	0	1	4	4	4	2	3	3
		0	1	2	3	0	1	1	1	4	2	2
Oldest page			0	1	2	3	0	0	0	1	4	4

Interactive Simulation Tool

- https://sim-50.github.io/cs-tools/

FIFO with 3 frames



FIFO Example 1



FIFO with 4 frames



Belady's anomaly

- Reduce the number of page faults by supplying more memory
  - Use previous reference string and FIFO algorithm
  - Add another page to physical memory (total 4 pages)
- More page faults (10 vs. 9), not fewer!
  - This is called *Belady's anomaly*
  - Adding more pages shouldn't result in worse performance!
- Motivated the study of paging algorithms

CLOCK Simulation

Modeling more replacement algorithms

- Paging system characterized by:
  - Reference string of executing process
  - Page replacement algorithm
  - Number of page frames available in physical memory (*m*)
- Model this by keeping track of all *n* pages referenced in array *M*
  - Top part of *M* has *m* pages in memory
  - Bottom part of *M* has *n-m* pages stored on disk
- Page replacement occurs when page moves from top to bottom
  - Top and bottom parts may be rearranged without causing movement between memory and disk

Example: LRU

- Model LRU replacement with
  - 8 unique references in the reference string
  - 4 pages of physical memory
- Array state over time shown below
- LRU treats list of pages like a stack

	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	1	3	4	1
	0	2	1	3	5	4	6	3	7	4	7	3	3	5	5	3	1	1	1	7	1	3	4	1
		0	2	1	3	5	4	6	3	7	4	7	7	3	3	5	3	3	3	1	7	1	3	4
			0	2	1	3	5	4	6	3	3	4	4	7	7	7	5	5	5	3	3	7	1	3
				0	2	1	3	5	4	6	6	6	6	4	4	4	7	7	7	5	5	5	7	7
					0	2	1	1	5	5	5	5	5	6	6	6	4	4	4	4	4	4	5	5
						0	2	2	1	1	1	1	1	1	1	1	6	6	6	6	6	6	6	6
							0	0	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Stack algorithms

- LRU is an example of a stack algorithm
- For stack algorithms
  - Any page in memory with *m* physical pages is also in memory with *m*+1 physical pages
  - Increasing memory size is guaranteed to reduce (or at least not increase) the number of page faults
- Stack algorithms do not suffer from Belady's anomaly
- *Distance* of a reference == position of the page in the stack before the reference was made
  - Distance is ∞ if no reference had been made before
  - Distance depends on reference string and paging algorithm: might be different for LRU and optimal (both stack algorithms)

Predicting page fault rates using distance

- Distance can be used to predict page fault rates
- Make a single pass over the reference string to generate the distance string on-the-fly
- Keep an array of counts
  - Entry *j* counts the number of times distance *j* occurs in the distance string
- The number of page faults for a memory of size *m* is the sum of the counts for *j*>*m*
  - This can be done in a single pass!
  - Makes for fast simulations of page replacement algorithms
- This is why virtual memory theorists like stack algorithms!

LRU



OPT

